Noir Atelier — AI-Native E-Commerce Platform: Unified Architecture & Implementation

Oct 29, 2026 · @Arffy technologies

Building one integrated system where AI agents and e-commerce customers share the same backend, database, tools, and business rules.

Unified Vision: One System, Not Two

The traditional approach is wrong: **Agent System First → Then Add E-Commerce.** This creates data fragmentation, duplicate business logic, and agents operating on mock data that never match the real storefront.

Noir Atelier's approach is different. We build one integrated **AI-native commerce platform** where:

- **Agents and customers use the same backend API endpoints.** When a customer searches for "black shirts," they hit the same `/api/products/search` endpoint an agent uses. Same validation, same data, same results.
- **Both read and write to the same database.** Orders created by customers are the same records agents process. Inventory levels are authoritative across both.
- **Business rules flow one direction.** Free shipping threshold, exchange window, GST rates, payment methods—defined once in `business_config` table, enforced everywhere.
- **Agent and customer decisions are audited together.** Refunds approved by an agent, returns initiated by a customer, payments verified by the system—all in the same `audit_log`.

Three User Interfaces, One Backend

Interface	Users	Purpose
Storefront (React/Next.js)	Customers	Browse, search, add to cart, checkout, track orders

Interface	Users	Purpose
Agent API (LangGraph + Claude)	AI agents (internal)	Catalog search, cart mgmt, order processing, returns, support
Admin Portal (React)	Staff	Business config, manual orders, reporting, escalations

All three hit the same 32 tables. Same inventory. Same pricing. Same customer records.

Development Implication: Build in Lockstep

Phase 1 (Weeks 1–4) is NOT "build the agent system," then "add a storefront." It's simultaneously:

- Backend (FastAPI) with all service layers
- Agent framework (LangGraph) with tool definitions pointing to backend APIs
- Storefront UI (React) also calling the same APIs

By the end of Phase 1, both agents and customers can browse products, because they're both calling `/api/products/search` with the same validation, pagination, and filters.

The Cost of Failure

If built separately:

- Agents hallucinate prices, inventory, customer data that differ from what the UI shows
- Business config gets hardcoded in agents and in the backend, drifting over time
- Refund logic exists in both agent code and order service, creating contradictions
- Testing requires mocking the backend twice (once for agents, once for UI)

One system avoids all of this. Agents are a client of the backend, just like the React app.

Architecture Overview: Shared Backend

Three-Layer Stack

Layer	Technology	Shared?
Frontend	React/Next.js (Storefront), React (Admin Portal)	No — agents use API directly

Layer	Technology	Shared?
API Gateway & Services	FastAPI, LangGraph (agent orchestration)	YES — both agents and UI call these
Data	PostgreSQL (32 tables), Redis (cache), Qdrant (vectors), Elasticsearch (search)	YES — single source of truth

API Contract: One Interface, Two Clients

When a customer searches for black shirts via the React storefront, they make:

```
GET /api/products/search?q=black%20shirt&category=clothing&page=1&limit=20
Authorization: Bearer <customer-token>
```

When an agent needs to search the same catalog for a customer inquiry, it makes:

```
GET /api/products/search?q=black%20shirt&category=clothing&page=1&limit=20
Authorization: Bearer <agent-service-account-token>
```

Same endpoint. Same validation. Same response shape. The difference is in the authorization token (customer vs. agent) and which business rules apply (store hours visibility, regional pricing, inventory filters).

Core Service Layers

1. **Product Service** (`/api/products/*`)
   - Search, filter, get details, fetch recommendations
   - Draws from PostgreSQL (main catalog) + Elasticsearch (full-text search) + Qdrant (vector embeddings for "similar items")
   - Both agents and the React storefront call these endpoints
2. **Order Service** (`/api/orders/*`)
   - Create order, list customer orders, get order status, apply coupon
   - Writes to `orders` , `order_items` , `payments` tables
   - Agents call this to place orders on customer's behalf; UI calls this for customer's own orders
3. **Cart Service** (`/api/cart/*`)
   - Add item, remove item, get cart, checkout
   - Session or user-specific; agents maintain a cart per conversation session

- UI maintains a cart per logged-in customer

4. **Customer Service** (`/api/customers/*`)
   - Get profile, update address, list past orders, preferences
   - Agents call this to look up customer context during conversation
   - UI calls this for account management
5. **Returns & Refunds** (`/api/returns/*`)
   - Initiate return, track status, approve refund
   - Both agents and UI can initiate; business rules prevent agent-approval of high-value returns
6. **Business Config Service** (`/api/config/*`)
   - Get active policies (free shipping threshold, exchange window, GST rates, mock GSTIN, payment methods)
   - Called at startup by both UI and agent to ensure alignment
   - Cached in Redis; updated by admin portal

Authorization & Data Access

Actor	Token Type	API Access	DB Access	Can Write
Customer (UI)	OAuth/JWT	`/api/products`, `/api/orders`, `/api/cart`, `/api/customers/{me}`	Read: customers, products, orders; Write: orders, cart	Orders, addresses, preferences
Agent (Internal)	Service Account	All `/api/*` endpoints + `/api/internal/*` for audit	Read: all tables; Write: orders, refunds, audit_log	Orders, refunds, audit entries
Admin (Portal)	Admin Token	All endpoints + `/api/admin/*`	Full read/write	All tables

Data Flow: Cart → Order → Fulfillment

```
Customer/Agent            API Gateway              Service Layer
Database
__________________        ___________              _____________
________

[POST /cart/add]  ⟶    [Auth token]      ⟶    [Cart Service]    ⟶
[cart table]
                          [Validate scope]         [Check inventory]

[POST /orders]    ⟶    [Auth token]      ⟶    [Order Service]   ⟶
[orders]
                          [Check totals]           [Calc tax/ship]
[order_items]
                          [Verify customer]        [Call Payment]
[payments]
                                                   [Log audit event]

[GET /orders/{id}] ⟶   [Auth token]      ⟶    [Order Service]   ⟶
[orders]
                          [Check ownership]        [Join items]
[shipments]
                                                   [Get tracking]

[audit_log]
```

Why This Matters

When an agent processes a refund and writes `audit_log.reason = "Customer dissatisfied"`, the admin who reviews it sees the exact same record the customer sees in their order history. There is no "agent version" of the refund and a "customer version" — one source of truth across the entire system.

When business config updates (free shipping threshold changes from ₹1,999 to ₹1,500), both the React storefront and the agent system see the change within seconds (Redis TTL). No hardcoded values to hunt down.

Agent Specifications: 9 Agents on Real Data

Each agent operates on the same database, same API endpoints, and same business rules. None are mocked or isolated.

1. Catalog Agent

- **Purpose**: Answer product questions, search, recommend similar items
- **Triggers**: "What black shirts do you have?", "Show me linen kurtas under ₹2,000"
- **API Calls**: `/api/products/search`, `/api/products/{id}`, `/api/products/{id}/recommendations`
- **Data**: Reads from `products`, `product_images`, `inventory`, `categories`, vector embeddings in Qdrant
- **Output**: Structured product cards with images, price, stock, rating

2. Cart & Checkout Agent

- **Purpose**: Add items to cart, calculate totals, apply discounts, proceed to checkout
- **Triggers**: "Add this to my cart", "What's my total?", "Apply code SUMMER20"
- **API Calls**: `/api/cart/add`, `/api/cart/get`, `/api/orders/calculate`, `/api/coupons/validate`
- **Data**: Reads/writes `cart` (session-specific), reads `coupons`, `business_config` (for free shipping threshold)
- **Output**: Updated cart with itemized breakdown, discounts, shipping, GST

3. Order Processing Agent

- **Purpose**: Create confirmed orders, handle payment, trigger fulfillment
- **Triggers**: "Complete my purchase", initiated after checkout agent
- **API Calls**: `/api/orders`, `/api/payments/initiate`, `/api/shipments/schedule`
- **Data**: Writes `orders`, `order_items`, `payments`; reads `business_config`, `customers`
- **Constraints**: Cannot refund orders > ₹50,000 without admin approval
- **Output**: Order confirmation with order number, ETA, tracking link

4. Fulfillment Agent

- **Purpose**: Generate packing slips, update inventory, schedule pickups, confirm dispatch
- **Triggers**: Order placed, inventory check, pickup request
- **API Calls**: `/api/shipments/{id}`, `/api/inventory/reserve`, `/api/logistics/schedule`
- **Data**: Reads/writes `shipments`, `inventory`, `orders`; reads `addresses`
- **Integration**: Syncs with mock logistics API (no real integration in Phase 1)
- **Output**: Packing slip PDF, shipment label, tracking number

5. Customer Support Agent

- **Purpose**: Answer FAQs, troubleshoot orders, escalate to humans
- **Triggers**: "Where's my order?", "I haven't received a refund", "Can I exchange this?"
- **API Calls**: `/api/orders/search`, `/api/refunds/list`, `/api/returns/initiate`, `/api/support/escalate`
- **Data**: Reads `orders`, `order_items`, `payments`, `refunds`, `returns`, `audit_log`; writes `support_tickets`
- **Constraints**: Escalates orders over 30 days old, high-value disputes, customers with 3+ complaints
- **Output**: Status update, next steps, or human support ticket

6. Returns & Refunds Agent

- **Purpose**: Initiate returns, approve refunds within policy, restock items
- **Triggers**: "I want to return this", "Process my refund"
- **API Calls**: `/api/returns`, `/api/refunds`, `/api/inventory/restock`, `/api/payments/refund`
- **Data**: Reads/writes `returns`, `refunds`, `order_items`, `inventory`; reads `business_config` (exchange window)
- **Constraints**: Cannot refund outside exchange_window_days (7) without manager approval; cannot exceed GST rules
- **Output**: Return shipping label, refund status, restock confirmation

7. Inventory Management Agent

- **Purpose**: Monitor stock levels, flag low stock, suggest restocking, archive discontinued items
- **Triggers**: Scheduled daily, or on demand ("Low stock alert for Saree SKU-001?")
- **API Calls**: `/api/inventory/list`, `/api/inventory/update`, `/api/products/discontinue`, `/api/audit/log`
- **Data**: Reads/writes `inventory`, `audit_log`; reads `products`, `orders`
- **Integration**: Syncs with supplier data (mock in Phase 1)
- **Output**: Restocking recommendations, discontinued items log

8. Analytics Agent

- **Purpose**: Answer business questions about sales, trends, customer behavior
- **Triggers**: "Top sellers this month?", "Customer retention rate?", "Revenue by category?"

- **API Calls**: `/api/orders/analytics`, `/api/customers/analytics`, `/api/products/analytics`
- **Data**: Reads (only) `orders`, `order_items`, `customers`, `products`, `audit_log`
- **Output**: Charts, tables, summaries (no writes; read-only)

9. Configuration & Policy Agent

- **Purpose**: Manage business rules, tax rates, shipping zones, payment methods
- **Triggers**: Admin only — "Update free shipping to ₹1,500", "Add new payment method"
- **API Calls**: `/api/admin/config`, `/api/admin/policies`, `/api/admin/audit`
- **Data**: Reads/writes `business_config`, `policies`, `payment_methods`; logs all changes in `audit_log`
- **Constraints**: Admin-only; all changes tracked for compliance
- **Output**: Confirmation of policy update with audit trail

Agent Lifecycle Per Conversation

1. **Init**: Customer starts chat → Catalog Agent springs up, ready to search
2. **Search**: Catalog Agent finds items, customer adds one → Cart Agent loads
3. **Checkout**: Cart Agent calculates totals → Order Processing Agent awaits payment confirmation
4. **Fulfillment**: Payment succeeds → Fulfillment Agent schedules pickup, issues tracking
5. **Post-Sale**: Order > 7 days old + no refund → Support Agent can escalate; if return requested → Returns Agent handles
6. **Background**: Inventory Agent runs daily; Analytics Agent answers manager questions anytime

All 9 agents read/write to the **same database**. No duplication. No async queues that get out of sync. Every agent action is logged to `audit_log` with timestamp, actor, change, and reason.

API Design: Shared Endpoints

All endpoints return JSON. Authentication via Bearer token (OAuth for customers, service account for agents/admin). All endpoints validate input, enforce authorization, and log to `audit_log`.

Product Endpoints (`/api/products/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`GET`	`/api/products/search`	Full-text + vector search	Yes	Yes	Public
`GET`	`/api/products/{id}`	Product details + reviews + stock	Yes	Yes	Public
`GET`	`/api/products/{id}/recommendations`	Similar items using vectors	Yes	Yes	Public
`GET`	`/api/categories`	Browse categories + counts	Yes	Yes	Public
`POST`	`/api/products`	Create new product	Admin	No	Admin
`PUT`	`/api/products/{id}`	Update product (name, desc, price)	Admin	No	Admin
`DELETE`	`/api/products/{id}`	Soft-delete product	Admin	No	Admin

Cart Endpoints (`/api/cart/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`POST`	`/api/cart/add`	Add item to session cart	Yes	Yes	Customer or Service Account
`DELETE`	`/api/cart/remove/{item_id}`	Remove item from cart	Yes	Yes	Customer or Service Account
`GET`	`/api/cart`	Get current cart (with totals)	Yes	Yes	Customer or Service Account
`POST`	`/api/cart/clear`	Clear entire cart	Yes	Yes	Customer or Service Account

Order Endpoints (`/api/orders/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`POST`	`/api/orders`	Create order from cart	Yes	Yes	Customer or Service Account
`GET`	`/api/orders`	List my orders (filtered by customer_id)	Yes	Yes	Customer or Service Account
`GET`	`/api/orders/{id}`	Get order details + items + shipment	Yes	Yes	Customer or Service Account
`POST`	`/api/orders/{id}/cancel`	Cancel order (only if not shipped)	Yes	Yes	Customer or Service Account
`GET`	`/api/orders/analytics`	Sales analytics (dashboard queries)	Yes	No	Agent or Admin

Returns & Refunds (`/api/returns/*` , `/api/refunds/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`POST`	`/api/returns`	Initiate return (within exchange_window_days)	Yes	Yes	Customer or Service Account
`GET`	`/api/returns/{id}`	Check return status + shipping label	Yes	Yes	Customer or Service Account
`POST`	`/api/refunds/{id}/approve`	Approve refund (agent only for <₹50k)	Yes	No	Service Account or Admin
`POST`	`/api/refunds/{id}/process`	Execute refund to payment method	Yes	No	Service Account or Admin
`GET`	`/api/refunds`	List all refunds (admin dashboard)	No	No	Admin

Business Config (`/api/config/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`GET`	`/api/config/policies`	Get active policies (free shipping threshold, exchange window, GST %, mock GSTIN, payment methods)	Yes	Yes	Public (cached)
`PUT`	`/api/admin/config/policies`	Update policies	No	No	Admin

Method	Endpoint	Purpose	Agents	UI	Auth
`GET`	`/api/config/payment-methods`	List active payment methods	Yes	Yes	Public

Customer Endpoints (`/api/customers/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`POST`	`/api/customers/register`	Create new customer	Yes (on behalf)	Yes	Public
`GET`	`/api/customers/{id}`	Get customer profile	Yes	Yes	Owner or Agent
`PUT`	`/api/customers/{id}`	Update profile (name, email, phone)	Yes	Yes	Owner or Agent
`GET`	`/api/customers/{id}/addresses`	List saved addresses	Yes	Yes	Owner or Agent
`POST`	`/api/customers/{id}/addresses`	Add new address	Yes	Yes	Owner or Agent
`GET`	`/api/customers/{id}/preferences`	Notification preferences, referrals	Yes	Yes	Owner or Agent

Support & Escalation (`/api/support/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`POST`	`/api/support/escalate`	Create support ticket (agent-initiated)	Yes	No	Service Account
`GET`	`/api/support/tickets`	List support tickets	No	No	Admin
`POST`	`/api/support/tickets/{id}/resolve`	Mark ticket resolved	No	No	Admin

Admin/Audit (`/api/admin/*` , `/api/audit/*`)

Method	Endpoint	Purpose	Agents	UI	Auth
`GET`	`/api/audit/log`	Access audit log (filtered by actor, resource, date)	Yes	No	Admin or Service Account
`POST`	`/api/admin/inventory/sync`	Trigger inventory sync with supplier (mock)	No	No	Admin

Method	Endpoint	Purpose	Agents	UI	Auth
`GET`	`/api/admin/dashboard`	KPI dashboard (revenue, orders, returns, customers)	No	No	Admin

Example Request & Response

```
POST /api/orders
Authorization: Bearer eyJhbGc...
Content-Type: application/json

{
  "customer_id": "cust_0001",
  "items": [
    {"product_id": "prod_saree_001", "quantity": 1, "price_at_purchase":
4999}
  ],
  "shipping_address_id": "addr_0001",
  "payment_method": "upi",
  "coupon_code": "SUMMER20"
}

--- Response (201 Created) ---

{
  "order_id": "ord_0001",
  "customer_id": "cust_0001",
  "status": "confirmed",
  "items": [
    {
      "product_id": "prod_saree_001",
      "product_name": "Black Silk Saree",
      "quantity": 1,
      "price_per_unit": 4999,
      "gst": 899.82,
      "total": 5898.82
    }
  ],
  "subtotal": 4999,
  "discount": -249.95,
  "shipping": 0,
  "gst_total": 649.87,
  "grand_total": 5398.92,
```

```
  "payment_status": "pending",
  "created_at": "2026-10-29T14:30:00Z",
  "estimated_delivery": "2026-11-02",
  "tracking_url": "https://noir.test/track/ord_0001"
}
```

Response Codes

Code	Meaning	Example
`200`	Success (GET, PUT)	Product fetched
`201`	Created (POST)	Order created
`400`	Bad request	Invalid coupon code
`401`	Unauthorized	Missing/expired token
`403`	Forbidden	Customer trying to refund another's order
`404`	Not found	Product ID doesn't exist
`422`	Unprocessable (validation)	Cart total < 0, invalid address
`500`	Server error	Database connection failed

All error responses include a `code` string (e.g., `"invalid_coupon"`) and a `message` for display to user (agent or customer).

Database Model: 32 Tables, One Truth

Core Tables (Shared by Agents & UI)

Customers (auth & profiles)

- `customers` (customer_id, email, phone, name, registered_at, last_login, status)
- `customer_addresses` (address_id, customer_id, street, city, zip, is_primary)
- `customer_preferences` (pref_id, customer_id, email_notifications, sms_notifications, newsletter, referral_code)

Products & Inventory

- `products` (product_id, sku, name, description, price, category_id, created_at, is_active, soft_deleted)

- `product_images` (image_id, product_id, url, alt_text, display_order)
- `categories` (category_id, name, parent_id, icon_url)
- `inventory` (inventory_id, product_id, quantity_available, quantity_reserved, quantity_damaged, last_sync)
- `product_vectors` (vector_id, product_id, embedding, updated_at) — for similarity search

Shopping & Orders

- `cart` (cart_id, customer_id, session_id, product_id, quantity, created_at, expires_at)
- `orders` (order_id, customer_id, order_date, status, subtotal, tax_amount, shipping_fee, total, currency)
- `order_items` (item_id, order_id, product_id, product_name, quantity, price_at_purchase, gst_applied)
- `coupons` (coupon_id, code, discount_type, discount_value, min_order_value, max_uses, uses_remaining, valid_from, valid_until)
- `coupon_usage` (usage_id, coupon_id, order_id, customer_id, discount_applied)

Payments

- `payments` (payment_id, order_id, method, status, amount, transaction_id, gateway_response, created_at)
- `payment_methods` (method_id, name, is_active, min_order, max_order, icon_url) — card, upi, bank transfer, etc.

Fulfillment

- `shipments` (shipment_id, order_id, carrier, tracking_number, status, scheduled_pickup, shipped_date, delivered_date)
- `shipping_zones` (zone_id, name, countries, base_fee, per_kg_fee)

Returns & Refunds

- `returns` (return_id, order_id, reason, initiated_date, status, return_shipping_label_url)
- `refunds` (refund_id, return_id, amount, reason, approved_by, approved_date, processed_date, payment_refund_id)

Support

- `support_tickets` (ticket_id, customer_id, order_id, subject, description, priority, status, created_by, assigned_to, resolved_at)

Audit & Compliance

- `audit_log` (log_id, timestamp, actor_type [customer|agent|admin], actor_id, resource_type [order|return|refund|product], resource_id, action [create|update|refund_approve], change_details, reason)
- `compliance_flags` (flag_id, order_id, flag_type [high_value|fraud_suspected|gst_issue], severity, created_at, resolved_at)
- `fraud_flags` (fraud_id, customer_id, flag_type [multiple_returns|chargeback|high_frequency], severity, created_at)

Business Configuration

- `business_config` (config_id, config_key [free_shipping_threshold, exchange_window_days, gst_rate, mock_gstin, etc.], config_value, data_type, updated_at, updated_by)
  - Example rows:
    - `free_shipping_threshold_inr` → `1999`
    - `exchange_window_days` → `7`
    - `gst_rate` → `0.18`
    - `mock_gstin` → `"33MOCKA1234Z1Z5"`
    - `default_currency` → `"INR"`
    - `active_payment_methods` → `["upi","card","bank_transfer"]`

Agent Authorization

- `agent_permissions` (perm_id, agent_type [catalog|cart|order_processing|etc.], resource_type [orders|refunds|inventory], action [read|write|admin], created_at)
  - Catalog Agent → read on products, inventory, vectors
  - Order Processing Agent → write on orders, payments; read on business_config, customers
  - Returns Agent → write on returns, refunds; read on business_config, orders; cannot write if order > 30 days old

Data Access Patterns

How Customers Query (via React UI)

```
GET /api/products/search?q=black%20shirt
  → SELECT * FROM products WHERE name ILIKE '%black%shirt%' AND
is_active=true
  → JOIN with inventory for stock
  → JOIN with product_images for thumbnails
```

How Agents Query (via LangGraph + Claude)

```
Agent: "Search for black shirts under 2000"
  → Catalog Agent calls /api/products/search?
q=black&max_price=2000&category=shirts
  → Same endpoint, same validation, same response
```

Order Creation (Agent or Customer)

```
1. Agent/UI POSTs /api/orders with cart items
2. Order Service
   a) Reads from business_config (GST rate, free shipping threshold)
   b) Calculates tax = subtotal * gst_rate
   c) Calculates shipping (0 if subtotal >= free_shipping_threshold)
   d) INSERTs into orders, order_items
   e) INSERTS into audit_log (actor=customer_id or agent_type,
action="create_order")
3. Payment Service
   a) Calls external payment gateway (mocked in Phase 1)
   b) UPDATEs payments.status
4. Fulfillment Service (async)
   a) Reads from inventory, orders
   b) INSERTs shipment record
   c) UPDATEs audit_log
```

Return Initiation (Agent Negotiates, Customer Approves)

```
1. Customer initiates return via UI (or agent on customer's behalf)
   → INSERT into returns, UPDATEs orders.status → pending_return
2. Returns Agent checks business_config.exchange_window_days
   → If order.order_date < NOW - 7 days → escalate to support
   → Otherwise → auto-approve
3. Returns Agent
   a) INSERTs return_shipping_label_url
   b) INSERTs refund record
   c) UPDATEs inventory.quantity_available
   d) UPDATEs audit_log (actor=agent_type, reason="policy_compliant_return")
```

Why Shared Database Works

1. **No Duplication**: Product data exists once. Inventory levels are live for both UI and agents.
2. **Business Rules Centralized**: Free shipping threshold stored once in business_config; both UI and agents read it.
3. **Audit Trail Complete**: Every agent action (refund approval, inventory update) goes to audit_log with timestamp and reason.
4. **Concurrency Safe**: PostgreSQL ACID transactions; agents and customers can transact simultaneously without conflicts.
5. **Scalability Ready**: Design supports soft deletes, partitioning by order_date, and sharding on customer_id for future growth.

Indexes for Performance

Table	Index	Purpose
`products`	(category_id, is_active, price)	Browse by category, filter by price
`inventory`	(product_id)	Quick stock lookup
`orders`	(customer_id, order_date DESC)	Customer's order history
`audit_log`	(actor_type, actor_id, timestamp DESC)	Compliance queries
`returns`	(order_id, initiated_date DESC)	Return status dashboard
`product_vectors`	(embedding)	Vector similarity search (Qdrant)

Caching Strategy

Redis caches:

- `business_config:*` (TTL: 1 hour) — ensures both agent and UI see same policies
- `products:{id}` (TTL: 30 min) — product details
- `inventory:{product_id}` (TTL: 5 min) — live stock (short TTL for accuracy)
- `customer:{id}:addresses` (TTL: 10 min) — address list

Cache invalidation on write (cache delete on POST/PUT/DELETE).

Business Configuration: One Source of Truth

All business policies live in the `business_config` table and Redis cache. When the admin changes a policy, both agents and UI see the change within seconds. No hardcoded values. No version drift.

Core Policies

```
{
  "store": {
    "name": "Noir Atelier",
    "currency": "INR",
    "timezone": "Asia/Kolkata",
    "phone": "+91-XXXXXXXXXX",
    "email": "support@noir.test"
```

```
  },

  "shipping": {
    "free_shipping_threshold_inr": 1999,
    "default_shipping_fee": 99,
    "standard_delivery_days": 3,
    "express_delivery_days": 1,
    "express_fee": 199,
    "zones": [
      {"name": "Metro", "fee": 99, "days": 2},
      {"name": "Tier2", "fee": 149, "days": 3},
      {"name": "Tier3", "fee": 249, "days": 5}
    ]
  },

  "returns_and_exchanges": {
    "exchange_window_days": 7,
    "return_window_days": 30,
    "restocking_fee_percent": 0,
    "return_reason_categories": ["size_mismatch", "color_mismatch",
"quality_issue", "damaged", "changed_mind"],
    "auto_approve_returns_under_value_inr": 2500,
    "require_approval_above_value_inr": 5000
  },

  "tax": {
    "gst_rate": 0.18,
    "gst_rate_category_overrides": {
      "textiles": 0.05,
      "jewelry": 0.12
    },
    "mock_gstin": "33MOCKA1234Z1Z5",
    "tax_invoice_on": "all_orders"
  },

  "payments": {
    "active_payment_methods": ["upi", "card", "bank_transfer", "emi"],
    "min_order_for_emi": 5000,
    "emi_partners": ["hdfc_bank", "icici_bank"],
    "card_networks": ["visa", "mastercard", "amex"],
    "mock_payment_success_rate": 0.95,
    "payment_timeout_seconds": 300
  },
```

```
  "discounts": {
    "max_discount_percent": 50,
    "max_coupon_discount_inr": 1000,
    "bulk_discount_threshold_qty": 5,
    "bulk_discount_percent": 10,
    "referral_bonus_inr": 500
  },

  "customer": {
    "allow_guest_checkout": false,
    "email_verification_required": true,
    "password_min_length": 8,
    "session_timeout_minutes": 30
  },

  "agent_restrictions": {
    "agent_can_auto_refund_upto_inr": 50000,
    "agent_can_escalate_orders_older_than_days": 30,
    "agent_can_initiate_refund_without_return": false,
    "agent_rate_limit_requests_per_minute": 100
  },

  "demo_mode": {
    "enabled": true,
    "mock_products": [
      {"sku": "SKU-001", "name": "Black Silk Saree", "price": 4999, "stock":
50},
      {"sku": "SKU-002", "name": "Handloom Cotton Kurta", "price": 1999,
"stock": 30},
      {"sku": "SKU-003", "name": "Gold Jewelry Set", "price": 15999, "stock":
5},
      {"sku": "SKU-004", "name": "Linen Dupatta", "price": 899, "stock":
100},
      {"sku": "SKU-005", "name": "Embroidered Blouse Piece", "price": 1299,
"stock": 20}
    ],
    "mock_customers": [
      {"customer_id": "cust_demo_001", "name": "Arjun Kumar", "email":
"arjun@noir.test", "phone": "+91-9999000001"},
      {"customer_id": "cust_demo_002", "name": "Priya Sharma", "email":
"priya@noir.test", "phone": "+91-9999000002"}
    ],
```

```
    "simulated_payment_success": true,
    "demo_expires_at": "2026-12-31T23:59:59Z"
  },

  "compliance": {
    "collect_billing_address": true,
    "collect_pan": false,
    "collect_aadhaar": false,
    "terms_and_conditions_version": "2.0",
    "privacy_policy_version": "2.0"
  }
}
```

How Agents Use Business Config

Catalog Agent (at startup)

```
GET /api/config/policies
  → Reads product_images, categories, pricing (no override)
  → Shows same products to customers as to itself
```

Cart Agent (on adding item)

```
Item price: ₹4,999
Subtotal: ₹4,999
GET /api/config/policies
  → Reads gst_rate = 0.18
  → Reads free_shipping_threshold_inr = 1999
  → Since 4999 > 1999 → shipping_fee = 0
  → gst = 4999 * 0.18 = 899.82
  → Total = 4999 + 899.82 = 5898.82
```

Order Processing Agent (on payment success)

```
Order created with gst = 899.82, shipping = 0
  → Reads mock_gstin from business_config
  → Stores in order.invoice_gst_number = "33MOCKA1234Z1Z5"
  → Matches what customer sees on invoice PDF
```

Returns Agent (on return initiation)

```
Order order_date = 2026-10-22
Today = 2026-10-29
Days since order = 7

GET /api/config/policies
  → Reads exchange_window_days = 7
  → 7 <= 7 → Still within window
  → Reads auto_approve_returns_under_value_inr = 2500
  → Order value = 5398.92 > 2500
  → Escalate to manager approval
```

How UI Uses Business Config

Storefront (on product page)

```
Product price: ₹4,999
Cart total with this item: ₹4,999

GET /api/config/policies
  → Reads free_shipping_threshold_inr = 1999
  → Shows "Free Shipping" badge (since total > 1999)
  → Reads gst_rate = 0.18
  → Shows tax breakdown in cart
```

Checkout (before submit)

```
Cart items: [
  {sku: "SKU-001", qty: 1, price: 4999}
]
Subtotal: ₹4,999
Shipping: ₹0 (free, exceeds threshold)
Tax: ₹899.82 (18% GST)
Total: ₹5,898.82

GET /api/config/policies
  → Shows active_payment_methods = [upi, card, bank_transfer, emi]
  → Since total (5898.82) > min_order_for_emi (5000) → Show EMI option
```

Admin Portal (policy updates)

```
Admin changes: free_shipping_threshold_inr: 1999 → 1500

1. PUT /api/admin/config/policies
2. Service writes to business_config table + audit_log
3. Redis cache invalidated (cache key: business_config:*)
4. Agent & UI fetch fresh config on next request
5. Both now enforce ₹1,500 threshold (was ₹1,999)
```

Why This Matters

1. **No Hardcoded Values**: Policy changes don't require code deployment
2. **Agent-UI Consistency**: Both read the same policies from the same table
3. **Audit Trail**: Every policy change logged with timestamp + admin user
4. **Demo Mode Enabled**: Phase 1 uses demo data; can flip to production mode without code changes
5. **Compliance Ready**: GST rules, GSTIN, tax calculations stored centrally for auditing

Workflows: 6 Real End-to-End Flows

Workflow 1: Search & Browse (Catalog Agent + Customer UI)

```
Customer opens Noir Atelier app
└─ GET /api/products/categories
   (UI shows: Sarees, Kurtas, Jewelry, Duppatas)

Customer: "Show me black sarees under ₹5000"
└─ Catalog Agent calls /api/products/search
   {
     "q": "black sarees",
     "category": "sarees",
     "max_price": 5000
   }
└─ Backend: Full-text search (Elasticsearch) + vector similarity (Qdrant)
   └─ JOINs products, inventory, product_images
   └─ Returns: [
        {sku: "SKU-001", name: "Black Silk Saree", price: 4999, stock: 50,
rating: 4.8, images: [...]}
      ]

Agent shows customer 3 options
Customer clicks SKU-001
```

```
└─ GET /api/products/SKU-001
   (Agent fetches full details: description, care, reviews, similar items)
└─ GET /api/products/SKU-001/recommendations
   (Shows 5 similar sarees using vector embeddings)
```

Workflow 2: Add to Cart & Review (Cart Agent + Customer UI)

```
Customer: "Add this to my cart"
└─ POST /api/cart/add
   {
     "product_id": "prod_saree_001",
     "quantity": 1
   }
   └─ Backend:
      a) Checks inventory.quantity_available > 0
      b) Reserves quantity: inventory.quantity_reserved += 1
      c) INSERTs into cart table
      d) UPDATEs Redis: cache expires in 5 min
└─ Returns: {item_id, product_name, price: 4999, total: 5898.82 (including
18% GST)}

Customer: "What's my total?"
└─ GET /api/cart
   └─ Backend:
      a) READs business_config.free_shipping_threshold_inr = 1999
      b) Cart subtotal = 4999 > 1999 → shipping = 0
      c) READs business_config.gst_rate = 0.18
      d) gst_amount = 4999 * 0.18 = 899.82
      e) total = 4999 + 0 + 899.82 = 5898.82
   └─ Returns:
      {
        items: [{
          product_id: "prod_saree_001",
          quantity: 1,
          price_per_unit: 4999,
          gst: 899.82
        }],
        subtotal: 4999,
        shipping: 0,
        tax: 899.82,
        total: 5898.82,
        free_shipping_applied: true
      }
```

```
Customer: "Have you got any discount codes?"
└─ Agent offers: "Try SUMMER20 for 5% off"
Customer: "Apply SUMMER20"
└─ POST /api/coupons/validate
   {
     "code": "SUMMER20",
     "cart_total": 5898.82
   }
   └─ Backend:
      a) Checks coupons table: code = "SUMMER20" exists, discount_type =
"percent", discount_value = 5
      b) Checks min_order_value: 5898.82 > 1000 ✔
      c) Checks uses_remaining: > 0 ✔
      d) Calculates discount = 5898.82 * 0.05 = 294.94
   └─ Returns: {discount: 294.94, new_total: 5603.88}
```

Workflow 3: Checkout & Payment (Order Processing Agent)

```
Customer: "Complete my purchase"
└─ POST /api/orders
   {
     "customer_id": "cust_0001",
     "cart_items": [{
       "product_id": "prod_saree_001",
       "quantity": 1,
       "price_at_purchase": 4999
     }],
     "shipping_address_id": "addr_0001",
     "payment_method": "upi",
     "coupon_code": "SUMMER20"
   }

   Backend Order Processing Agent:
   a) LockS inventory row (SELECT ... FOR UPDATE)
   b) READs business_config (gst_rate, free_shipping_threshold, mock_gstin)
   c) Calculates:
      - subtotal = 4999
      - discount = 294.94
      - shipping = 0 (free)
      - gst = 4999 * 0.18 = 899.82
      - total = 5603.88
   d) INSERTs order: {order_id, customer_id, status: "pending_payment",
```

```
total: 5603.88}
   e) INSERTs order_items
   f) DECREMENTS inventory.quantity_reserved, INCREMENTS quantity_available
(net: 0)
   g) UPDATEs coupon_usage table
   h) INSERTs audit_log: {actor: "cust_0001", action: "create_order",
resource: "ord_0001"}
   i) Calls payment gateway (mocked):
      POST /mock-gateway/charge
        {amount: 5603.88, method: "upi", order_id: "ord_0001"}
   j) Receives success: {transaction_id: "txn_xyz123"}
   k) INSERTs payment: {payment_id, order_id, status: "success", amount:
5603.88, transaction_id: "txn_xyz123"}
   l) UPDATEs order.status = "confirmed"
   m) INSERTs audit_log: {actor: "payment_gateway", action:
"payment_success", resource: "ord_0001"}
└─ Returns:
      {
        order_id: "ord_0001",
        status: "confirmed",
        items: [...],
        total: 5603.88,
        payment_status: "success",
        estimated_delivery: "2026-11-02",
        tracking_url: "https://noir.test/track/ord_0001"
      }

Agent shows customer confirmation page
Customer receives order confirmation email
```

Workflow 4: Fulfillment (Fulfillment Agent, Background)

```
On order.status = "confirmed" (async trigger)
└─ Fulfillment Agent wakes up
   a) GETs order details
   b) UPDATEs inventory.quantity_available -= 1 (final deduction)
   c) INSERTs shipment: {order_id, status: "picking", scheduled_pickup: NOW +
24 hours}
   d) Calls mock logistics API:
      POST /logistics/schedule-pickup
        {order_id, address, items_count: 1, weight_kg: 2.5}
   e) Receives: {tracking_number: "TRK123456789"}
   f) UPDATEs shipment.tracking_number, status = "packed"
   g) INSERTs audit_log: {actor: "fulfillment_agent", action:
"shipment_created", resource: "ord_0001"}
   h) UPDATEs order.status = "shipped"

Customer gets SMS: "Your order is on the way. Track:
https://noir.test/track/ord_0001"
```

Workflow 5: Return & Refund (Returns Agent Negotiates)

```
Customer (day 5 after order): "I want to return this saree"
└─ Agent checks business_config.exchange_window_days = 7
   Day 5 <= 7 → Within window

Customer: "Color isn't what I expected"
└■ POST /api/returns
   {
     "order_id": "ord_0001",
     "reason": "color_mismatch",
     "customer_message": "Looks different on my screen"
   }
   └─ Backend:
      a) INSERTs return: {order_id, reason, initiated_date: NOW, status:
"created"}
      b) UPDATEs order.status = "pending_return"
      c) Generates return_shipping_label
      d) INSERTs audit_log: {actor: "cust_0001", action: "initiate_return",
resource: "ret_0001"}
   └─ Returns: {return_id, shipping_label_url, instructions}

Agent sends customer shipping label
Customer ships saree back
```

```
Returns Agent (day 7): Receives return, inspects quality
└─ POST /api/refunds/ret_0001/approve
   {
     "amount": 5603.88,
     "reason": "quality_verified"
   }
   └─ Backend (admin/agent only):
      a) Checks business_config.auto_approve_returns_under_value_inr = 2500
      b) Return amount 5603.88 > 2500 → Requires manager approval
      c) Escalates to support_tickets (status: "pending_approval")
      d) Manager reviews, approves
      e) INSERTs refund: {return_id, amount, reason, approved_by:
"manager_001", approved_date: NOW}
      f) POST /api/payments/refund
         {transaction_id: "txn_xyz123", amount: 5603.88}
      g) Payment gateway confirms refund
      h) UPDATEs refund.status = "processed", payment_refund_id:
"rfnd_abc456"
      i) INCREMENTS inventory.quantity_available += 1 (restock)
      j) UPDATEs order.status = "refunded"
      k) INSERTs audit_log: {actor: "manager_001", action: "refund_approved",
resource: "ord_0001", reason: "policy_verified"}

Customer gets SMS: "Refund of ₹5,603.88 processed. Expect it in 2-3 days."
```

Workflow 6: Analytics (Analytics Agent, Admin Only)

```
Admin dashboard query: "What were our top sellers this month?"
└─ Analytics Agent calls /api/orders/analytics
   {
     "group_by": "product_id",
     "aggregate": "quantity_sum",
     "date_from": "2026-10-01",
     "date_to": "2026-10-31"
   }
   └─ Backend:
      SELECT product_id, product_name, SUM(quantity) AS units_sold,
SUM(order_items.price_at_purchase * quantity) AS revenue
      FROM order_items
      JOIN orders ON order_items.order_id = orders.order_id
      JOIN products ON order_items.product_id = products.product_id
      WHERE orders.order_date BETWEEN '2026-10-01' AND '2026-10-31'
      GROUP BY product_id
```

```
      ORDER BY units_sold DESC
      LIMIT 10;
   └─ Returns: [
        {product_id: "prod_saree_001", product_name: "Black Silk Saree",
units_sold: 12, revenue: 59987},
        ...
      ]

Admin also checks: "Customer retention rate (repeat buyers) this month?"
└─ SELECT COUNT(DISTINCT customer_id) AS repeat_customers
   FROM orders
   WHERE customer_id IN (
     SELECT customer_id FROM orders
     WHERE order_date < '2026-10-01'
   )
   AND order_date BETWEEN '2026-10-01' AND '2026-10-31';
└─ Result: 23 repeat customers, 15% retention rate

Admin dashboard displays: Top 10 products, revenue trends, retention metrics,
refund rate
```

Key Points

1. **Same Data**: All workflows read from and write to the same PostgreSQL database.
2. **Shared APIs**: Cart Agent and Customer UI both call `/api/cart` endpoints.
3. **Consistent Rules**: Free shipping threshold, GST rate, exchange window enforced the same way everywhere.
4. **Complete Audit Trail**: Every action logged with actor (customer_id, agent_type, admin_id), timestamp, and reason.
5. **No Data Duplication**: One order record, one inventory row, one customer record—no syncing between "agent version" and "customer version."

Phase Roadmap: 5 Phases Over 20 Weeks

Development is **simultaneous**, not sequential. Backend APIs are built in lockstep with agent tools and UI components that consume them.

Phase 1: Foundation (Weeks 1–4) — Core Shopping Flow

Deliverables:

- **Backend API**:

  - Auth (OAuth for customers, service accounts for agents/admin)
  - Product search (`/api/products/search` , `/api/products/{id}`)
  - Cart management (`/api/cart/*`)
  - Order creation (`/api/orders`)
  - Business config service (`/api/config/policies`)
  - Inventory checks (read-only)
  - Basic audit logging to `audit_log` table
- **Agent Framework**:
  - LangGraph setup with Claude backend
  - Catalog Agent: search, get product details, recommendations
  - Cart Agent: add to cart, calculate totals, apply coupons
  - Order Processing Agent: create order, initiate payment
  - Mock payment gateway (fake success rate 95%)
  - Agent tool definitions (schema validation, atomic operations)
- **Storefront UI** (React/Next.js):
  - Homepage with category browse
  - Product search & filter
  - Product detail page (images, reviews, stock)
  - Shopping cart page (itemized, totals, discount input)
  - Checkout page (address, payment method selection)
  - Order confirmation page
  - Demo mode: 5 mock products, 2 mock customers
- **Database**:
  - 32-table schema (normalized, with FK/PK constraints)
  - business_config table with demo values
  - Seed data: 5 products, 10 categories, demo customers
  - Indexes for product search, inventory lookup, order history
  - Redis setup for business_config caching

Testing: Both agents and UI can browse, add to cart, and complete a purchase using the same API endpoints.

Success Criterion: Customer and agent can both buy a saree end-to-end; order appears in same database.

Phase 2: Monetization (Weeks 5–8) — Payments, Returns, Compliance

Deliverables:

- **Backend API**:
  - Real payment gateway integration (Razorpay or Stripe mock)
  - Refund API (`/api/refunds/*`)
  - Returns flow (`/api/returns/*`)
  - Business config for tax (GST, mock GSTIN)
  - Tax calculation service (reads from business_config)
  - Compliance flag system (audit_log enhancements)
  - Support escalation API
- **Agent Framework**:
  - Returns & Refunds Agent: initiate, approve (with constraints)
  - Customer Support Agent: FAQ, escalations, order status
  - Tax calculation in Order Agent (reads business_config.gst_rate)
  - Refund constraint: agent can approve < ₹50k; > needs manager
  - Support ticket creation on agent escalation
- **Storefront UI**:
  - Order history page (with return/refund buttons)
  - Return initiation flow (reason dropdown, upload images)
  - Refund status tracking
  - Support ticket form (customer-initiated escalations)
  - Tax breakdown on invoices
  - Multiple payment methods (card, UPI, bank transfer, EMI)
- **Database**:
  - Expand business_config: tax rates, GST rules, exchange window, mock GSTIN
  - compliance_flags table (high-value orders, suspicious activity)
  - fraud_flags table (multiple returns, chargebacks)
  - Soft delete strategy (orders never deleted, only flagged)
  - Audit log expanded to capture reason for all changes

Testing: Agent and customer can both initiate returns, refunds process correctly, tax calculations match on both sides.

Success Criterion: End-to-end order + return + refund; ₹5,000 refund requires manager approval (constraint enforced).

Phase 3: Fulfillment & Inventory (Weeks 9–12) — Logistics, Stock Management

Deliverables:

- **Backend API**:
  - Shipment tracking (`/api/shipments/*`)
  - Inventory sync (`/api/inventory/reserve` , `/api/inventory/restock`)
  - Fulfillment webhooks (order placed → scheduling picks up)
  - Logistics integration (mock carrier API)
  - Inventory analytics (low stock alerts, turnover)
- **Agent Framework**:
  - Fulfillment Agent: packing, shipping label generation, tracking updates
  - Inventory Management Agent: stock level monitoring, restocking recommendations
  - Integration with mock logistics partner (scheduled pickups, tracking)
- **Storefront UI**:
  - Track shipment page (carrier info, last mile tracking)
  - Inventory display on product pages (stock level, estimated delivery)
  - Admin inventory dashboard (stock levels, low-stock alerts)
  - Packing list downloads (admin)
- **Database**:
  - Shipments table: full lifecycle (picking, packed, in_transit, delivered)
  - Inventory table: quantity_available, quantity_reserved, quantity_damaged
  - Fulfillment audit entries (packing notes, quality checks)
  - Soft deletes on discontinued products

Testing: Order placed → Fulfillment Agent schedules pickup within 24h → Tracking updates visible to customer.

Success Criterion: Complete order-to-delivery flow; stock reserved and released correctly; no overselling.

Phase 4: Advanced Agents & Analytics (Weeks 13–16) — Personalization, Reporting

Deliverables:

- **Backend API**:
  - Recommendation engine (`/api/products/recommendations`)
  - Customer analytics (`/api/customers/analytics`)
  - Order analytics (`/api/orders/analytics`)
  - Personalization API (customer preferences, browsing history)
  - Admin dashboard API (KPI endpoint)
- **Agent Framework**:
  - Analytics Agent: sales reports, retention rates, top products
  - Personalization: agents remember customer preferences within conversation
  - Vector embeddings (Qdrant) for product similarity
  - Offer suggestions ("customers who bought X also bought Y")
  - Churn prediction (identify customers at risk of not returning)
- **Storefront UI**:
  - Admin dashboard (revenue, orders, returns, customer metrics)
  - Personalized product recommendations (on homepage, in email)
  - Customer review system (agents can ask for reviews)
- **Database**:
  - Customer_vectors table (embedding of purchase history for similarity)
  - Recommendations table (pre-computed top-selling combos)
  - Analytics views (materialized views for fast dashboard queries)

Testing: Admin dashboard shows accurate KPIs; agents suggest personalized products; recommendations match customer behavior.

Success Criterion: Analytics Agent can answer "Top 10 sellers last month?" in < 2 seconds.

Phase 5: Scale & Compliance (Weeks 17–20) — Production Hardening

Deliverables:

- **Backend API**:
  - Rate limiting (per agent, per customer)
  - Comprehensive audit logging
  - Data export for compliance (GST records, invoices)
  - Webhook retries (payment, shipment updates)

    - Multi-tenancy prep (if expanding to other brands)
- **Agent Framework**:
    - Configuration Agent: admin-only policy updates
    - Rate limiting: agents max 100 requests/min
    - Agent permission matrix (who can refund, escalate, etc.)
    - Error recovery (graceful degradation if payment gateway down)
    - Multi-turn conversation memory (agents remember past interactions)
- **Storefront UI**:
    - Mobile optimization (PWA)
    - Accessibility (WCAG AA)
    - Dark mode
    - Performance optimizations (lazy loading, image optimization)
- **Database**:
    - Partitioning by order_date (weekly or monthly)
    - Sharding strategy prepared (customer_id as shard key)
    - Backup & disaster recovery tests
    - Read replicas for analytics queries
    - PIPL/GDPR compliance: data retention policies, customer export
- **Deployment**:
    - CI/CD pipeline (GitHub Actions)
    - Staging environment mirrors production
    - Feature flags for gradual rollouts
    - Monitoring & alerting (response times, error rates, payment failures)
    - Documentation (API docs, runbooks, postmortems)

Testing: Load test with 1000 concurrent users; stress test payment gateway failures; compliance audit with sample data export.

Success Criterion: System runs 24/7 with < 0.1% error rate; all business rules auditable; ready for production launch.

Cross-Phase Dependencies

Dependency	Introduced	Used By
Business config table	Phase 1	All phases (GST, shipping, exchange window)
Audit log	Phase 1	All phases (compliance, debugging)
Agent permissions	Phase 1	Phase 5 (enforcement at API level)
Inventory reserve/release	Phase 1	Phase 3 (fulfillment integration)
Vector embeddings	Phase 4	Phase 4 (recommendations)
Analytics views	Phase 4	Phase 5 (monitoring dashboards)

Team Allocation (Example)

- **Backend** (2 engineers): FastAPI service layer, database schema, API design
- **Agents** (1 engineer): LangGraph, tool definitions, agent workflows
- **Frontend** (1 engineer): React, UI components, customer-facing flows
- **DevOps** (1 engineer): Deployment, monitoring, databases (Postgres, Redis, Qdrant)
- **QA** (1 engineer): Test automation, agent testing, payment flow validation

Handoff Protocol

Each phase ends with:

1. **Integration test**: Agents and UI both use completed APIs; results compare
2. **Compliance review**: Audit log captures all actions; business rules enforced
3. **Retrospective**: What slowed us down? Better estimates next phase?

Development Setup: Mock Mode & Local Running

Prerequisites

- **Backend**: Python 3.11+, FastAPI, SQLAlchemy
- **Database**: PostgreSQL 14+, Redis 7+, Qdrant vector DB, Elasticsearch 8+
- **Agents**: Claude API key, LangGraph, Python
- **Frontend**: Node.js 18+, React 18, Next.js
- **DevOps**: Docker, Docker Compose (optional but recommended)

Quick Start (Demo Mode)

1. **Clone repo & install dependencies**

```
git clone https://github.com/noir-atelier/platform.git
cd platform

# Backend
cd backend && pip install -r requirements.txt

# Agents
pip install langgraph anthropic

# Frontend
cd ../frontend && npm install
```

2. **Configure environment**

```
# .env.local
ANTHROPIC_API_KEY=sk-...
DATABASE_URL=postgresql://user:pass@localhost/noir
REDIS_URL=redis://localhost:6379
QDRANT_URL=http://localhost:6333
DEMO_MODE=true
```

3. **Start services (Docker Compose)**

```
docker-compose up -d postgres redis qdrant elasticsearch
```

4. **Initialize database**

```
cd backend
alembic upgrade head  # Apply migrations
python scripts/seed_demo.py  # Load 5 products, 2 customers, demo policies
```

5. **Start backend API**

```
cd backend
uvicorn app.main:app --reload --port 8000
# API running on http://localhost:8000
# Docs on http://localhost:8000/docs (Swagger)
```

6. **Start agents (separate terminal)**

```
cd agents
python -m langgraph.server --port 8001
# Or run individually:
python agents/catalog_agent.py
python agents/cart_agent.py
# (In Phase 1, agents run in-process; Phase 5 uses agent server)
```

7. **Start frontend UI**

```
cd frontend
npm run dev
# UI running on http://localhost:3000
```

Test the Flow (Manual)

```
# 1. API health check
curl http://localhost:8000/health
# {"status": "ok", "demo_mode": true}

# 2. Search products (via API directly)
curl "http://localhost:8000/api/products/search?q=saree"
# [
#   {
#     "product_id": "prod_saree_001",
#     "sku": "SKU-001",
#     "name": "Black Silk Saree",
#     "price": 4999,
#     "stock": 50,
#     "images": [...]
#   }
# ]

# 3. Search products (via agent)
python -c "
import sys; sys.path.insert(0, '.');
from agents.catalog_agent import search_products;
print(search_products('black sarees under 5000'))
"
# Same results

# 4. Add to cart (API)
curl -X POST http://localhost:8000/api/cart/add \
  -H 'Content-Type: application/json' \
```

```
  -H 'Authorization: Bearer demo_cust_001' \
  -d '{"product_id": "prod_saree_001", "quantity": 1}'
# {"item_id": "cart_1", "product_name": "Black Silk Saree", "price": 4999}

# 5. Place order
curl -X POST http://localhost:8000/api/orders \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer demo_cust_001' \
  -d '{
    "customer_id": "cust_demo_001",
    "items": [{"product_id": "prod_saree_001", "quantity": 1}],
    "shipping_address_id": "addr_demo_001",
    "payment_method": "upi",
    "coupon_code": null
  }'
# {"order_id": "ord_001", "status": "confirmed", "total": 5898.82, ...}
```

Demo Data (Seeded on Init)

Products

```
SKU-001 | Black Silk Saree | 4999 | stock: 50
SKU-002 | Handloom Cotton Kurta | 1999 | stock: 30
SKU-003 | Gold Jewelry Set | 15999 | stock: 5
SKU-004 | Linen Dupatta | 899 | stock: 100
SKU-005 | Embroidered Blouse Piece | 1299 | stock: 20
```

Customers

```
cust_demo_001 | Arjun Kumar | arjun@noir.test
cust_demo_002 | Priya Sharma | priya@noir.test
```

Business Config (Demo Values)

```
free_shipping_threshold_inr: 1999
exchange_window_days: 7
gst_rate: 0.18
mock_gstin: "33MOCKA1234Z1Z5"
active_payment_methods: ["upi", "card", "bank_transfer"]
mock_payment_success_rate: 0.95
```

Automated Testing

```
# Run all tests
cd backend
pytest tests/ -v

# Test agents
pytest tests/agents/ -v

# Test APIs
pytest tests/api/ -v

# Integration test: agent + API + database
pytest tests/integration/test_end_to_end.py -v
```

Example Integration Test

```
# tests/integration/test_end_to_end.py

def test_customer_buys_saree_via_ui_and_agent():
    """
    Verify that a customer buying via UI and an agent buying on behalf
    both write to the same database and see the same data.
    """
    # 1. Agent searches
    results = catalog_agent.search_products("black sarees under 5000")
    assert len(results) > 0
    saree_id = results[0]["product_id"]

    # 2. Customer searches via API (same endpoint)
    response = client.get("/api/products/search?q=black%20sarees")
    assert response.status_code == 200
    api_results = response.json()
    assert api_results[0]["product_id"] == saree_id  # Same product

    # 3. Agent adds to cart and creates order
    agent_order = order_processing_agent.create_order(
        customer_id="cust_demo_001",
        items=[{"product_id": saree_id, "quantity": 1}]
    )

    # 4. Verify order in database
    from app.database import Session
    db_order = Session().query(Order).filter_by(
        order_id=agent_order["order_id"]
```

```
    ).first()
    assert db_order is not None
    assert db_order.customer_id == "cust_demo_001"
    assert db_order.total > 0

    # 5. Customer queries their order via API
    response = client.get(
        f"/api/orders/{agent_order['order_id']}",
        headers={"Authorization": "Bearer demo_cust_001"}
    )
    assert response.status_code == 200
    api_order = response.json()

    # 6. Verify API and database show same order
    assert api_order["order_id"] == agent_order["order_id"]
    assert api_order["total"] == db_order.total
    assert api_order["status"] == db_order.status

    # 7. Check audit log
    audit_entries = Session().query(AuditLog).filter_by(
        resource_id=agent_order["order_id"]
    ).all()
    assert len(audit_entries) >= 2  # order created + payment processed
    assert audit_entries[0].actor_type == "agent"
```

Debugging

Agent not calling API?

```
# Check agent logs
tail -f logs/agents.log

# Test agent tool directly
python -c "
from agents.catalog_agent import tools
for tool in tools:
    print(f'{tool.name}: {tool.description}')
"
```

API returning 401 Unauthorized?

```
# Check token in headers
curl -H 'Authorization: Bearer demo_cust_001' http://localhost:8000/api/cart

# Generate valid token
python -c "from app.auth import generate_token;
print(generate_token('cust_demo_001'))"
```

Product not showing in search?

```
# Check Elasticsearch index
curl http://localhost:9200/products/_search

# Rebuild index
python scripts/rebuild_search_index.py
```

Inventory Mismatch?

```
# Check inventory table
psql -U user -d noir -c "SELECT * FROM inventory WHERE product_id =
'prod_saree_001';"

# Check for reservations
psql -U user -d noir -c "SELECT SUM(quantity_reserved) FROM inventory;"
```

Next Steps After Setup

1. **Phase 1 Development**:
   - Build Product Service API
   - Build Cart Service API
   - Build Order Service API
   - Implement Catalog Agent
   - Implement Cart Agent
   - Implement Order Processing Agent
   - Build Storefront UI (product search, cart, checkout)
2. **Test Each Layer**:
   - Unit tests for business logic
   - Integration tests for API + database
   - Agent tests (tool calling, reasoning)
   - UI tests (component rendering, form submission)

3. **Manual QA**:
   - Buy a saree as a customer via UI
   - Have an agent buy the same saree
   - Verify both orders in audit_log with correct actors
4. **Deploy to Staging**:
   - Same schema and APIs, but with real payment gateway (test mode)
   - Invite early users to test
   - Gather feedback for Phase 2

Useful Commands

```
# Reset everything (dangerous!)
psql -U user -d noir -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
alembic upgrade head
python scripts/seed_demo.py

# Export demo data for sharing
pg_dump noir > noir_demo.sql

# Monitor live requests
watch -n 1 'tail -10 logs/api.log'

# Check agent reasoning
python -c "from agents.catalog_agent import agent; agent.verbose = True"
```